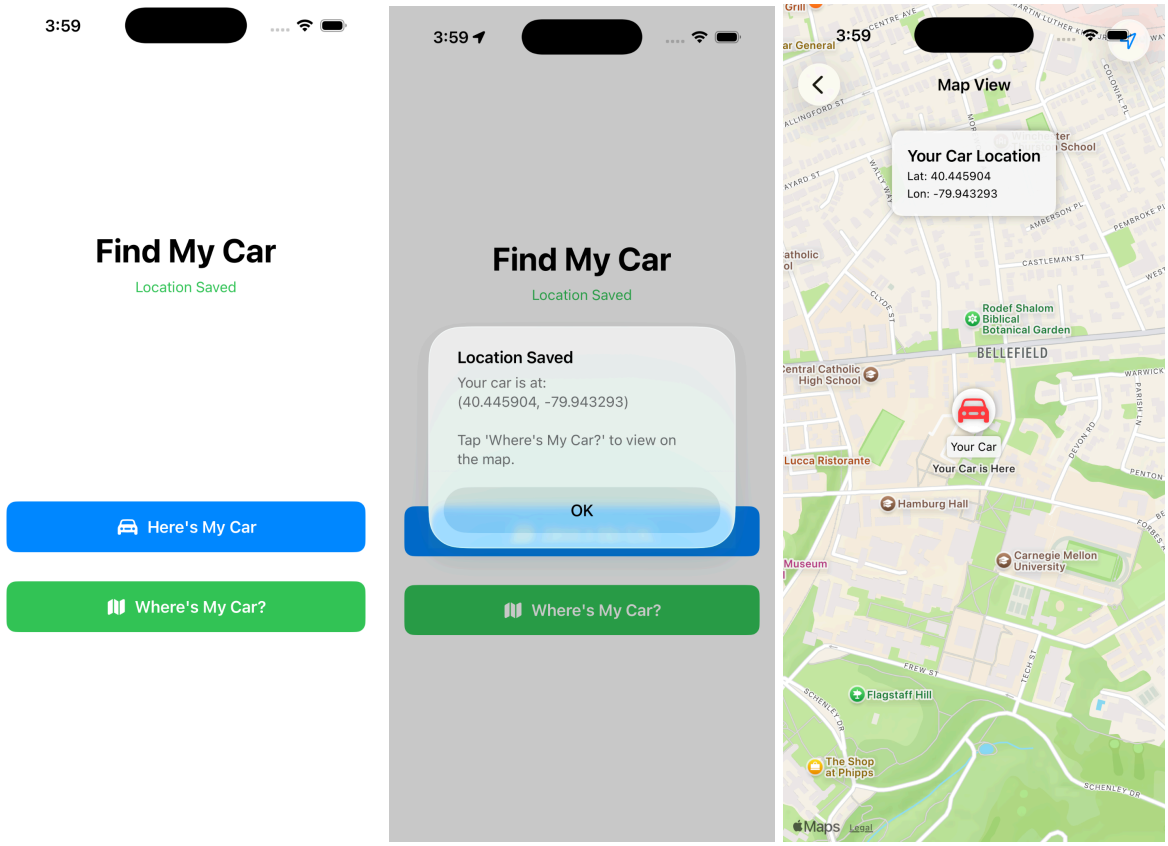
Lab 08: FindMyCar (React Native)

This week, we will make an app that allows users to track where their car is parked and where they are relative to the car. We'll use `react-native-maps` for the map, Expo's `Location` API for location services, and `AsyncStorage` for persistence. Here's a sneak peek of what the finished app looks something like:



The way this app works is that when we're ready to leave our car, we press the "Here's My Car" button and the location is saved. When we want to go back to the car, we press "Where's My Car" and it drops a marker on the map. (*Aside:* this was one of the first apps built for this class, back when it was Objective-C and there was no Swift yet -- and I actually used it regularly to help me keep track of where I parked. Apple's Maps app has since integrated this functionality, but it's still a great learning exercise for maps, locations, permissions, and simple persistence. 😉)

This lab will be built with Expo and TypeScript, the same setup we've used for the past several labs.

Note: Also, please do not forget to add comments to EVERY FUNCTION/METHOD IN YOUR CODE. Even the parts we give you! Good comments help you debug quicker, and also helps us assess your understanding.

Part 0: Setup

1. Create a new Expo project called `FindMyCarRN` and change into it:

```shell
npx create-expo-app@latest FindMyCarRN --template blank-typescript
cd FindMyCarRN
```

2. Install the packages we'll need for maps, location, persistence, and navigation:

```shell
npx expo install expo-location @react-native-async-storage/async-storage
react-native-maps @react-navigation/native @react-navigation/native-stack
react-native-screens react-native-safe-area-context @expo/vector-icons
```

   Icons come from `@expo/vector-icons`, which we just installed.

3. Create three folders inside your project: `models`, `viewModels`, and `views` -- same organizational pattern we've used since Lab 02.

Part 1: The Model

Create `models/CarCoordinate.ts`. In the Swift version this lab actually used **two** types -- a `Codable` `CarCoordinate` for persistence and an `Identifiable` `CarLocation` for map annotations. We don't need that split in React Native: `react-native-maps`'s `Marker` component accepts a plain `{ latitude, longitude }` object directly, so one type covers both jobs.

```typescript
export interface CarCoordinate {
  latitude: number;
  longitude: number;
}

export const defaultCarCoordinate: CarCoordinate = {
  latitude: 0.0,
  longitude: 0.0,
};

export function isValidCoordinate(coordinate: CarCoordinate): boolean {
```

```
  return coordinate.latitude !== 0 || coordinate.longitude !== 0;
}

export function formatCoordinate(coordinate: CarCoordinate): string {
  const lat = coordinate.latitude.toFixed(6);
  const lon = coordinate.longitude.toFixed(6);
  return `(${lat}, ${lon})`;
}
```

A few things worth calling out compared to the Swift version:

- **No `Codable` needed.** In Swift, `CarCoordinate: Codable, Equatable` was what let `PropertyListEncoder` serialize the coordinate to a plist file. Any object matching an interface can be serialized in TypeScript with `JSON.stringify()` -- no protocol conformance required.
- **Free-standing helpers instead of methods.** TypeScript interfaces can't hold behavior the way Swift structs can, so `isValidCoordinate` and `formatCoordinate` are exported as plain functions. Free functions are also easier to unit-test in isolation, which we'll take advantage of at the end of the lab.

Part 2: The Location View Model

As we've seen before, in Swift `LocationManager` was an `@Observable` class that wrapped `CLLocationManager` and used a delegate to receive push-based updates from Core Location. Expo's `expo-location` gives us the same functionality with a very different shape: instead of a delegate we implement, it's a set of `async` functions we call and `await` a result from -- much closer to a simple request/response than an ongoing subscription.

To keep the shared saved coordinate reactive across screens (so tapping "Here's My Car" on one screen updates any other screen reading it), we'll wrap it in the same Context + Provider + custom-hook pattern we've used since Lab 03.

Create `viewModels/LocationContext.tsx`:

TypeScript

```
import React, { createContext, useContext, useEffect, useState, ReactNode } from
'react';
import * as Location from 'expo-location';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { CarCoordinate, defaultCarCoordinate } from '../models/CarCoordinate';

const STORAGE_KEY = '@FindMyCar:coordinate';
```

```
type LocationContextValue = {
  coordinate: CarCoordinate;
  loading: boolean;
  saveCurrentLocation: () => Promise<CarCoordinate>;
  clearLocation: () => Promise<void>;
};

const LocationContext = createContext<LocationContextValue |
undefined>(undefined);

export function LocationProvider({ children }: { children: ReactNode }) {
  const [coordinate, setCoordinate] =
useState<CarCoordinate>(defaultCarCoordinate);
  const [loading, setLoading] = useState(true);

  // Load whatever coordinate was saved from a previous session on mount.
  // This is the RN equivalent of the Swift LocationManager's loadLocation()
  // being called from init().
  useEffect(() => {
    (async () => {
      try {
        const json = await AsyncStorage.getItem(STORAGE_KEY);
        if (json != null) setCoordinate(JSON.parse(json));
      } catch (error) {
        console.error('Error loading coordinate:', error);
      } finally {
        setLoading(false);
      }
    })();
  }, []);

  async function saveCurrentLocation(): Promise<CarCoordinate> {
    // Confirm services + permissions before asking for a position.
    if (!(await Location.hasServicesEnabledAsync())) {
      throw new Error('Location services are disabled.');
    }
    let { status } = await Location.getForegroundPermissionsAsync();
    if (status !== 'granted') {
      ({ status } = await Location.requestForegroundPermissionsAsync());
    }
    if (status !== 'granted') {
      throw new Error('Location access denied. Please enable in Settings.');
    }

    const fix = await Location.getCurrentPositionAsync({ accuracy:
Location.Accuracy.Highest });
    const next: CarCoordinate = {
      latitude: fix.coords.latitude,
```

```
      longitude: fix.coords.longitude,
    };
    await AsyncStorage.setItem(STORAGE_KEY, JSON.stringify(next));
    setCoordinate(next);
    return next;
  }

  async function clearLocation(): Promise<void> {
    await AsyncStorage.setItem(STORAGE_KEY, JSON.stringify(defaultCarCoordinate));
    setCoordinate(defaultCarCoordinate);
  }

  return (
    <LocationContext.Provider value={{ coordinate, loading, saveCurrentLocation,
clearLocation }}>
      {children}
    </LocationContext.Provider>
  );
}

export function useLocation(): LocationContextValue {
  const ctx = useContext(LocationContext);
  if (!ctx) throw new Error('useLocation must be used inside a LocationProvider');
  return ctx;
}
```

Understanding the code

- **`async`/`await` in place of a delegate.** In Swift, `saveCurrentLocation(completion:)` had to branch on `authorizationStatus`, kick off `startLocationUpdates()`, then wait for the `CLLocationManagerDelegate` callback `didUpdateLocations` to eventually fire with a result. Here, `getCurrentPositionAsync()` handles all of that internally and just resolves with a coordinate (or throws) when done. No delegate protocol, no completion handlers, no `NSObject` subclass -- just one linear `async` function reading top-to-bottom.
- **Same permission dance, different API.** We still check whether services are enabled and whether we have permission, requesting it if we don't. Expo returns the permission status as a simple string; we compare it against `'granted'` rather than switching over a `CLAuthorizationStatus` enum.
- **AsyncStorage in place of a plist.** The Swift version used `PropertyListEncoder` to write a `Coordinates.plist` file to the documents directory. RN doesn't have a plist

API, but `AsyncStorage` + `JSON.stringify` gets us the same asynchronous key/value persistence with less ceremony.

Before this will work, we need to configure the location permission. Open `app.json` and add these entries inside the `expo` object (merge with anything already there):

```JSON
"ios": {
  "supportsTablet": true,
  "infoPlist": {
    "NSLocationWhenInUseUsageDescription": "This app needs your location to save where you parked the car."
  }
},
"android": {
  "permissions": ["ACCESS_COARSE_LOCATION", "ACCESS_FINE_LOCATION"]
},
"plugins": [
  [
    "expo-location",
    {
      "locationWhenInUsePermission": "This app needs your location to save where you parked the car."
    }
  ]
]
```

This is Expo's config-plugin system wiring the permission strings into the native iOS/Android projects for us -- no Xcode Info-tab edits required.

Part 3: Navigation Setup

Create `types.ts` at the project root. Both screens need to share the saved coordinate, but since they both read it from the same `LocationContext`, the `Map` route doesn't need to carry it as a param -- keeping the param list empty avoids passing a possibly-stale copy:

```TypeScript
export type RootStackParamList = {
  Home: undefined;
  Map: undefined;
};
```

(Small design note vs. the older lab: last year, `AppView` passed a `coordinate` object through `navigation.navigate('Map', { coordinate })`. Because our context is the source of truth, we can skip that and let `MapView` just read the current value on its own. Fewer props to thread, no stale-snapshot risk.)

Part 4: The Views

Step 4.1: HomeView

Create `views/HomeView.tsx`. This is the two-button screen -- "Here's My Car" saves, "Where's My Car?" navigates to the map:

TypeScript

```
import React, { useState } from 'react';
import { View, Text, Pressable, Alert, ActivityIndicator, StyleSheet } from
'react-native';
import { Ionicons } from '@expo/vector-icons';
import { useNavigation } from '@react-navigation/native';
import type { NativeStackNavigationProp } from '@react-navigation/native-stack';
import { useLocation } from '../viewModels/LocationContext';
import { isValidCoordinate, formatCoordinate } from '../models/CarCoordinate';
import type { RootStackParamList } from '../types';

type NavProp = NativeStackNavigationProp<RootStackParamList, 'Home'>;

export default function HomeView() {
  const navigation = useNavigation<NavProp>();
  const { coordinate, saveCurrentLocation } = useLocation();
  const [busy, setBusy] = useState(false);

  const hasLocation = isValidCoordinate(coordinate);

  async function onSave() {
    if (busy) return;
    setBusy(true);
    try {
      const next = await saveCurrentLocation();
      Alert.alert(
        'Location Saved',
        `Your car is at:\n${formatCoordinate(next)}\n\nTap 'Where's My Car?' to
view on the map.`,
      );
    } catch (error) {
      const message = error instanceof Error ? error.message : 'Unknown error';
      Alert.alert('Location Error', message);
    } finally {
      setBusy(false);
    }
  }

  return (
```

```
    <View style={styles.container}>
      <View style={styles.titleBlock}>
        <Text style={styles.title}>Find My Car</Text>
        {hasLocation && <Text style={styles.subtitle}>Location Saved</Text>}
      </View>

      <Pressable style={[styles.button, styles.primary]} onPress={onSave}
disabled={busy}>
        {busy ? (
          <ActivityIndicator color="#fff" />
        ) : (
          <>
            <Ionicons name="car" size={22} color="#fff" style={styles.icon} />
            <Text style={styles.buttonText}>Here's My Car</Text>
          </>
        )}
      </Pressable>

      <Pressable
        style={[styles.button, hasLocation ? styles.success : styles.disabled]}
        onPress={() => navigation.navigate('Map')}
        disabled={!hasLocation}
      >
        <Ionicons name="map" size={22} color="#fff" style={styles.icon} />
        <Text style={styles.buttonText}>Where's My Car?</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', padding: 24, gap: 20 },
  titleBlock: { alignItems: 'center', marginBottom: 20 },
  title: { fontSize: 34, fontWeight: 'bold' },
  subtitle: { fontSize: 14, color: '#34C759', marginTop: 6 },
  button: { flexDirection: 'row', alignItems: 'center', justifyContent: 'center',
padding: 16, borderRadius: 10 },
  primary: { backgroundColor: '#007AFF' },
  success: { backgroundColor: '#34C759' },
  disabled: { backgroundColor: '#a1a1a1' },
  icon: { marginRight: 8 },
  buttonText: { color: '#fff', fontSize: 17, fontWeight: '600' },
});
```

A couple of things worth pointing out:

- **`useLocation()` gives us both the current coordinate and the save function** in a single hook call. Because it comes from context, tapping "Here's My Car" on this screen will also update `MapView` on its next render -- the same reactive behavior an `@Observable LocationManager` gave the Swift version.
- **`ActivityIndicator` while busy** replaces "Getting Location..." text -- looks native on both platforms without any extra work.
- **The "Where's My Car?" button is disabled** until there's a valid saved coordinate, mirroring the Swift version's `.disabled(...)` modifier on the `NavigationLink`.

Step 4.2: MapView

Create `views/MapView.tsx`. The Swift version used SwiftUI's `Map(position: $position)` with a custom `Annotation`; we'll use `react-native-maps`'s `MapView` + custom `Marker`, which have very close semantics:

TypeScript

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';
import MapView, { Marker, Region } from 'react-native-maps';
import { MaterialIcons } from '@expo/vector-icons';
import { useLocation } from '../viewModels/LocationContext';
import { formatCoordinate } from '../models/CarCoordinate';

export default function MapViewScreen() {
  const { coordinate } = useLocation();

  const initialRegion: Region = {
    latitude: coordinate.latitude,
    longitude: coordinate.longitude,
    latitudeDelta: 0.01,
    longitudeDelta: 0.01,
  };

  return (
    <View style={styles.container}>
      <MapView
        style={StyleSheet.absoluteFill}
        initialRegion={initialRegion}
        showsUserLocation
        showsMyLocationButton
        showsCompass
      >
        <Marker coordinate={coordinate} title="Your Car" description="Your car is
parked here">
```

```
          <View style={styles.markerContent}>
            <View style={styles.iconCircle}>
              <MaterialIcons name="directions-car" size={26} color="#FF3B30" />
            </View>
            <View style={styles.markerLabel}>
              <Text style={styles.markerLabelText}>Your Car</Text>
            </View>
          </View>
        </Marker>
      </MapView>

      <View style={styles.infoCard}>
        <Text style={styles.infoTitle}>Your Car Location</Text>
        <Text style={styles.infoText}>{formatCoordinate(coordinate)}</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1 },
  markerContent: { alignItems: 'center' },
  iconCircle: {
    padding: 6, backgroundColor: '#fff', borderRadius: 22,
    shadowColor: '#000', shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.2, shadowRadius: 2, elevation: 3,
  },
  markerLabel: {
    backgroundColor: 'rgba(255,255,255,0.9)', paddingHorizontal: 6,
paddingVertical: 2,
    borderRadius: 4, marginTop: 4,
  },
  markerLabelText: { fontSize: 11, fontWeight: '600' },
  infoCard: {
    position: 'absolute', top: 16, left: 16, right: 16,
    backgroundColor: 'rgba(255,255,255,0.95)', padding: 12, borderRadius: 10,
    shadowColor: '#000', shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.2, shadowRadius: 4, elevation: 4,
  },
  infoTitle: { fontSize: 16, fontWeight: '600', marginBottom: 2 },
  infoText: { fontSize: 13, color: '#333' },
});
```

A couple of parallels worth calling out:

- **`showsUserLocation` = `UserAnnotation()`.** Both drop the platform-native blue dot at the device's current position with zero setup on our end.
- **Custom `Marker` content = SwiftUI custom `Annotation`.** In both cases we supply our own view (car icon in a white circle, small label beneath) and the map framework renders it instead of the default pin.
- **`StyleSheet.absoluteFill`** on the MapView is the RN equivalent of the Swift version's `.ignoresSafeArea()` -- it fills the entire screen so the info card can float on top.

Step 4.3: Wiring It All Up

Open `App.tsx` and replace the boilerplate. `LocationProvider` wraps `NavigationContainer` so both screens share the same live coordinate:

```
TypeScript
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { LocationProvider } from './viewModels/LocationContext';
import HomeView from './views/HomeView';
import MapViewScreen from './views/MapView';
import type { RootStackParamList } from './types';

const Stack = createNativeStackNavigator<RootStackParamList>();

export default function App() {
  return (
    <LocationProvider>
      <NavigationContainer>
        <Stack.Navigator>
          <Stack.Screen name="Home" component={HomeView} options={{ title: 'Find
My Car' }} />
          <Stack.Screen name="Map" component={MapViewScreen} options={{ title:
'Map View' }} />
        </Stack.Navigator>
      </NavigationContainer>
    </LocationProvider>
  );
}
```

Run the app (`npx expo start`, then press `i` or `a`). Tap "Here's My Car", grant location permission when prompted, then tap "Where's My Car?" and confirm the map opens with a marker at the saved location.

To set a location in the iOS Simulator, use **Features → Location** and pick one (CMU is `40.444644, -79.945424`). For an Android emulator, use the "..." extended-controls panel's Location tab. You may occasionally get an error the first time you request a location in a simulator/emulator -- if so, background the app and relaunch it and it should clear up. Simulators are sometimes wonky about location. 🤷‍♂️

Part 5: Testing the Model

Our `CarCoordinate` module is pure logic (no React, no Expo, no AsyncStorage), which makes it exactly the kind of thing we should unit-test. That's why we pulled `isValidCoordinate` and `formatCoordinate` out as plain exported functions rather than baking them into the context.

Install Jest with the Expo preset (same setup as prior labs):

```shell
npx expo install jest-expo jest @types/jest --dev
```

Add a `jest` block and a `test` script to your `package.json`:

```json
"scripts": {
  "test": "jest"
},
"jest": {
  "preset": "jest-expo"
}
```

Also, open your `tsconfig.json` and add `"types": ["jest"]` under `compilerOptions` so TypeScript recognizes Jest's global testing functions (like `describe`, `test`, and `expect`):

```json
"compilerOptions": {
  "strict": true,
  "types": ["jest"]
}
```

Create a `__tests__` folder at the project root and add `carCoordinate.test.ts`:

TypeScript

```typescript
import {
  defaultCarCoordinate,
  isValidCoordinate,
  formatCoordinate,
  CarCoordinate,
} from '../models/CarCoordinate';

describe('CarCoordinate', () => {
  test('defaultCarCoordinate is (0, 0)', () => {
    expect(defaultCarCoordinate).toEqual({ latitude: 0, longitude: 0 });
  });

  test('isValidCoordinate returns false for the default', () => {
    expect(isValidCoordinate(defaultCarCoordinate)).toBe(false);
  });

  test('isValidCoordinate returns true when either component is non-zero', () => {
    expect(isValidCoordinate({ latitude: 40.4446, longitude: -79.9454
})).toBe(true);
    expect(isValidCoordinate({ latitude: 40.4446, longitude: 0 })).toBe(true);
    expect(isValidCoordinate({ latitude: 0, longitude: -79.9454 })).toBe(true);
  });

  test('formatCoordinate produces six-decimal output', () => {
    const c: CarCoordinate = { latitude: 40.4446443, longitude: -79.945424 };
    expect(formatCoordinate(c)).toBe('(40.444644, -79.945424)');
  });

  test('formatCoordinate pads short values to six decimals', () => {
    expect(formatCoordinate({ latitude: 1, longitude: 2 })).toBe('(1.000000,
2.000000)');
  });

  test('round-trips cleanly through JSON', () => {
    const original: CarCoordinate = { latitude: 40.4446, longitude: -79.9454 };
    const restored = JSON.parse(JSON.stringify(original));
    expect(restored).toEqual(original);
    expect(isValidCoordinate(restored)).toBe(true);
  });
});
```

Run it:

Shell

```shell
npm test
```

All six tests should pass. ✅

The JSON round-trip test is the RN equivalent of the Swift version's `PropertyListEncoder` / `PropertyListDecoder` round-trip test -- both are verifying that a coordinate can be serialized and rehydrated without data loss, using whichever encoding scheme the platform actually uses to persist it.

Why we don't test the provider directly: `LocationProvider` couples together React state, `expo-location`, and `AsyncStorage`. Testing it requires mocking all three and using `@testing-library/react-native`'s `renderHook`. That's more machinery than we need for a lab -- extracting the pure helpers got us the most valuable coverage with the least setup. If you want to go further, adding a mocked-permission test for the provider is a great optional exercise.

Wrapping Up

At this point, you have a working "find my car" app with real location services, persistent storage across app launches, and a real map with a custom marker. Not bad for one lab. Consider extending it with the following: (1) add a "Clear Location" button that calls `clearLocation()`, (2) show the distance between the user and the car on the map, or (if you're feeling ambitious) drop a `Polyline` connecting the two.

Qapla'